

```

#STORAGE REAL:18
#LOCAL REAL valpha, vbeta, vd_p, vq_p, vd_n, vq_n
#LOCAL REAL ialpha, ibeta, id_p, iq_p, id_n, iq_n
#LOCAL REAL valpha_p, vbeta_p, valpha_n, vbeta_n
#LOCAL REAL ialpha_p, ibeta_p, ialpha_n, ibeta_n
#LOCAL REAL sth, cth, sth_n, cth_n, theta, wp11, wint, verr
#LOCAL REAL wn_pll, kp_pll, ki_pll, wc_cc, kp_cc, ki_cc
#LOCAL REAL P_target, P_err, P_ramp, Q_target, Idref_p, Iqref_p
#LOCAL REAL ed_p, eq_p, ed_n, eq_n, vdpi_p, vqpi_p, vdpi_n, vqpi_n
#LOCAL REAL vdo_p, vqo_p, vdo_n, vqo_n, vdo_alpha, vdo_beta
#LOCAL REAL PIc, PI2, R3, w0, alpha_lpf, max_ramp

! =====
! INITIALIZATION BLOCK (Runs once at t=0)
! =====
#BEGIN
! FIXED: Use relative NSTORF indexing for multi-instance safety.
STORF(NSTORF) = 0.0
STORF(NSTORF+1) = 0.0
STORF(NSTORF+2) = 0.0
STORF(NSTORF+3) = 0.0
STORF(NSTORF+4) = 0.0
STORF(NSTORF+5) = 0.0
STORF(NSTORF+6) = 0.0
STORF(NSTORF+7) = 0.0
STORF(NSTORF+8) = 0.0
STORF(NSTORF+9) = 0.0
STORF(NSTORF+10) = 0.0
STORF(NSTORF+11) = 0.0
STORF(NSTORF+12) = 0.0
STORF(NSTORF+13) = 0.0
STORF(NSTORF+14) = 0.0
STORF(NSTORF+15) = 0.0
STORF(NSTORF+16) = 0.0
STORF(NSTORF+17) = 0.0
#ENDBEGIN

! =====
! PART 1: SYSTEM PARAMETERS, CLARKE TRANSFORMS, & SRF-PLL
! =====
PIc = 3.14159265358979
PI2 = 2.0*PIc
R3 = 1.7320508075688772
w0 = 2.0*3.14159*60.0

! SRF-PLL Gains (Tuned for weak grid stability, e.g., 10-50Hz)
wn_pll = PI2*$f_bw_pll
kp_pll = 2.0*$zeta_pll*wn_pll/$Vbase
ki_pll = (wn_pll*wn_pll)/$Vbase

! Recall PLL state variables from persistent memory
theta = STORF(NSTORF)
wint = STORF(NSTORF+1)

! Voltage Clarke Transformation (Amplitude-invariant mapping)
valpha = (2.0*$Vabc(1) - $Vabc(2) - $Vabc(3))/3.0
vbeta = ($Vabc(2) - $Vabc(3))/R3

! Low-pass filter for positive sequence voltage components
alpha_lpf = w0*DELTA/(1.0 + w0*DELTA)
STORF(NSTORF+7) = STORF(NSTORF+7) + alpha_lpf*(valpha - STORF(NSTORF+7))
STORF(NSTORF+8) = STORF(NSTORF+8) + alpha_lpf*(vbeta - STORF(NSTORF+8))
valpha_p = STORF(NSTORF+7)
vbeta_p = STORF(NSTORF+8)

```

```

! PLL Trigonometry
sth = SIN(theta)
cth = COS(theta)

! Positive sequence Park transformation (dq conversion)
vd_p = valpha_p*cth + vbeta_p*sth
vq_p = -valpha_p*sth + vbeta_p*cth

! PLL Loop Tracking Error (Forces vq_p strictly toward 0.0)
verr = -vq_p
wint = wint + ki_pll*verr*DELT
wpll = w0 + kp_pll*verr + wint

! Phase angle integration calculation loop with [0, 2*pi) wrapping
theta = theta + wpll*DELT
IF (theta .GE. PI2) theta = theta - PI2
IF (theta .LT. 0.0) theta = theta + PI2

! =====
! PART 2: POSITIVE SEQUENCE d-q CURRENT CONTROL
! =====

! Current Loop Gains (Tuned via IMC for <5ms response)
! Note: Set $fc_cc between 100Hz and 200Hz in component parameters
wc_cc = PI2*$fc_cc
kp_cc = wc_cc*$Lf
ki_cc = wc_cc*$Rf

! Calculate positive sequence feedback currents (Park Transform)
id_p = ialpha_p*cth + ibeta_p*sth
iq_p = -ialpha_p*sth + ibeta_p*cth

! Current tracking errors
! (Idref_p and Iqref_p will be provided by the Outer Loops in Parts 3 & 4)
ed_p = Idref_p - id_p
eq_p = Iqref_p - iq_p

! PI Controller Integration (using persistent memory slots 2 & 3)
STORF(NSTORF+2) = STORF(NSTORF+2) + ki_cc*ed_p*DELT
STORF(NSTORF+3) = STORF(NSTORF+3) + ki_cc*eq_p*DELT

! PI Controller Output
vdpi_p = kp_cc*ed_p + STORF(NSTORF+2)
vqpi_p = kp_cc*eq_p + STORF(NSTORF+3)

! Decoupling cross-matrix paths + forward-feeding equations
! Cancels out cross-coupling between d and q axes caused by the filter inductor
vdo_p = vd_p + vdpi_p - wpll*$Lf*iq_p
vqo_p = vq_p + vqpi_p + wpll*$Lf*id_p

! =====
! PART 3: ACTIVE POWER CONTROL & RAMP-RATE LIMITER
! =====

! Recall current active power ramp state from persistent memory
P_ramp = STORF(NSTORF+6)

! Active Power Ramp-Rate Limiter Setup
P_target = $Pref

! Calculate maximum allowed step size per simulation time step (DELT)
! $P_ramp_rate should be configured in your component parameters (e.g., MW/s)
max_ramp = $P_ramp_rate * DELT

! Calculate the requested step difference
P_err = P_target - P_ramp

```

```

! Apply hard limits to the step size to enforce the ramp rate
IF (P_err .GT. max_ramp) P_err = max_ramp
IF (P_err .LT. -max_ramp) P_err = -max_ramp

! Update the internal tracking variable for Active Power
P_ramp = P_ramp + P_err

! =====
! PART 4: REACTIVE POWER CONTROL & CURRENT REFERENCE GENERATION
! =====
! Reactive Control Selector Logic (0 = Direct Q, 1 = PF Mode)
IF ($PF_Mode .EQ. 0) THEN
! Mode 0: Direct Reactive Power Setpoint
Q_target = $Qref
ELSE
! Mode 1: Power Factor Control
! Added MIN/MAX bounds to prevent ACOS domain error (NaN crash)
IF (ABS($PF_setpoint) .GT. 0.01) THEN
Q_target = P_ramp * TAN(ACOS(MAX(-1.0, MIN(1.0, $PF_setpoint))))
ELSE
Q_target = 0.0
ENDIF
ENDIF

! Compute positive sequence target current references
! These values act as the setpoints for the PI current controllers in Part 2
IF (ABS(vd_p) .GT. 0.001) THEN
! Standard power-invariant inverse mapping
Idref_p = (2.0/3.0) * P_ramp / vd_p
Iqref_p = -(2.0/3.0) * Q_target / vd_p
ELSE
! Fault ride-through safety: Prevent division by zero during severe voltage dips
Idref_p = 0.0
Iqref_p = 0.0
ENDIF

! =====
! PART 5: NEGATIVE SEQUENCE CONTROL & WAVEFORM RECONSTRUCTION
! =====
! --- Negative Sequence Control Loop Operations ---
! Inverse Frame Mapping (Rotating in opposite direction)
sth_n = SIN(-theta)
cth_n = COS(-theta)

! Calculate negative sequence feedback currents
id_n = ialpha_n*cth_n + ibeta_n*sth_n
iq_n = -ialpha_n*sth_n + ibeta_n*cth_n

! Drive negative-sequence currents to 0.0 to cancel structural imbalance
ed_n = 0.0 - id_n
eq_n = 0.0 - iq_n

! PI Controller Integration (using persistent memory slots 4 & 5)
STORF(NSTORF+4) = STORF(NSTORF+4) + ki_cc*ed_n*DELT
STORF(NSTORF+5) = STORF(NSTORF+5) + ki_cc*eq_n*DELT

! PI Controller Output
vdpi_n = kp_cc*ed_n + STORF(NSTORF+4)
vqpi_n = kp_cc*eq_n + STORF(NSTORF+5)

! Internal negative-sequence decoupling math matrix
! FIXED: Added missing '$' prefix to Lf parameter
vd_n = valpha_n*cth_n + vbeta_n*sth_n

```

```

vq_n = -valpha_n*sth_n + vbeta_n*cth_n
vdo_n = vd_n + vdpi_n + wp11*$Lf*iq_n
vqo_n = vq_n + vqpi_n - wp11*$Lf*id_n

! =====
! 6. WAVEFORM RECONSTRUCTION & SIGNAL ASSIGNMENT
! =====

! Reconstruct mixed tracking arrays back into alpha-beta vectors
vdo_alpha = (vdo_p*cth - vqo_p*sth) + (vdo_n*cth_n - vqo_n*sth_n)
vdo_beta = (vdo_p*sth + vqo_p*cth) + (vdo_n*sth_n + vqo_n*cth_n)

! Inverse Clarke transformation outputs assigned to modulation ports
$Vmabc(1) = vdo_alpha
$Vmabc(2) = -0.5*vdo_alpha + (R3/2.0)*vdo_beta
$Vmabc(3) = -0.5*vdo_alpha - (R3/2.0)*vdo_beta

! Save dynamic runtime states back into persistent storage registers
STORF(NSTORF) = theta
STORF(NSTORF+1) = wint
STORF(NSTORF+6) = P_ramp

! Pass metrics out to the monitor nodes
$Theta = theta
$Fp11 = wp11 / PI2
$Wp11 = wp11
$P_out = P_ramp

! ADVANCE STORAGE POINTER FOR SYSTEM RUNTIME SIMULATION STEP
! This must happen exactly once at the very end of the component script
NSTORF = NSTORF + 18

```